Fourier Neural Operator: Learning Solution Operators in Spectral Space

Krishna Kumar

University of Texas at Austin

krishnak@utexas.edu

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

Learning Objectives

- Write a linear solution operator as a kernel, the Green's function
- See why translation invariance turns the kernel into a convolution
- Follow one Fourier wave through a convolution
- Learn the Fourier multipliers from input-output pairs
- Stack Fourier layers and activations into an FNO
- Train FNOs on Burgers' equation and Darcy flow

▸ Heat-flow intuition ▸ CNN filters and Fourier multipliers ▸ FNO

The loaded string

Here is a string pinned at both ends. A load $f(x)$ pushes on it, and its displacement $u(x)$ satisfies

$$-u''(x) = f(x), \qquad u(0) = u(1) = 0.$$

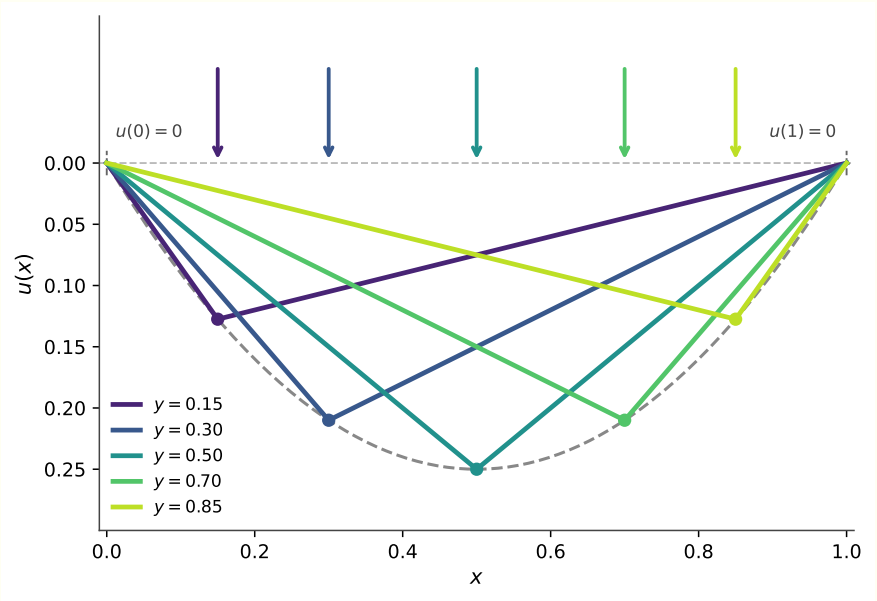
We want the whole map $f \mapsto u$, for every load at once.

The equation is local, because u'' at x depends only on u near x. The answer is not local, because a load anywhere bends the whole string. How does a load at one point move the rest of the string?

One point load

Put a unit load at one point y. Away from y there is no load, so $u'' = 0$ and the string is straight. The displacement is a tent with its peak under the load,

$$\kappa(x, y) = \begin{cases} x\,(1 - y), & x \le y, \\ y\,(1 - x), & x \ge y. \end{cases}$$



At $y = \dfrac{1}{2}$ the peak is $\dfrac{1}{4}$, and the slope drops by 1 at the load.

Adding up point loads

Any load is a row of small point loads. The piece of load near y has size $f(y)\,dy$, and it makes the tent $\kappa(x,y)\,f(y)\,dy$. Adding all the tents gives the displacement,

$$\boxed{u(x) = \int_0^1 \kappa(x,y)\,f(y)\,dy.}$$

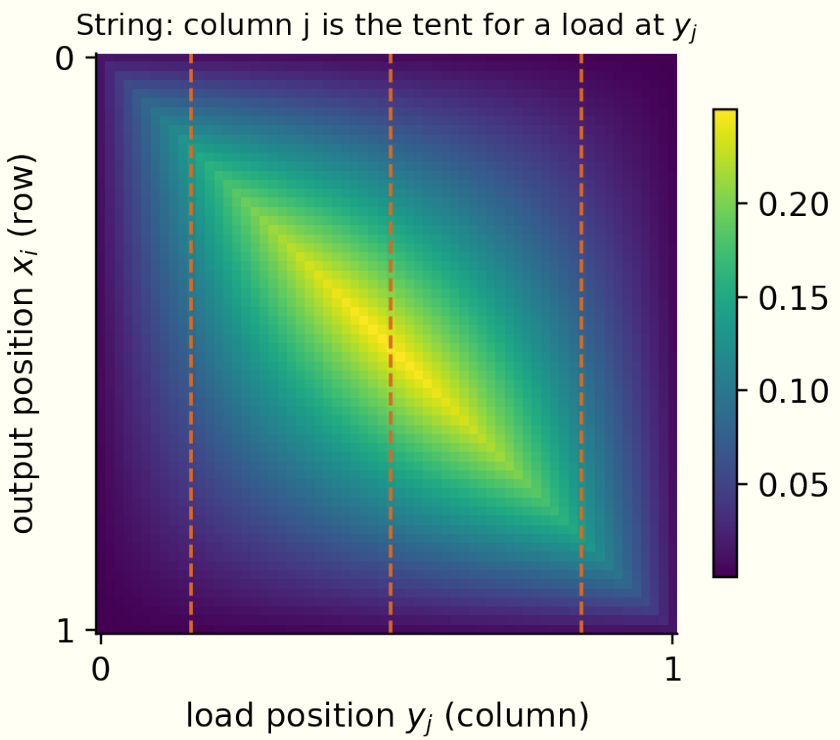
Check it with a uniform load $f = 1$. The integral gives $u = \dfrac{1}{2}\,x(1-x)$, and that parabola solves $-u'' = 1$.

The function $\kappa(x,y)$ is the **Green's function**. It is the kernel of the solution operator, the response at x to a unit load at y.

The Green's matrix

On a grid $x_1, \ldots, x_N$ with spacing h, the integral becomes a matrix times a vector,

$$u_i = \sum_{j=1}^{N} \underbrace{\kappa(x_i, y_j)\, h}_{K_{ij}}\ f_j.$$



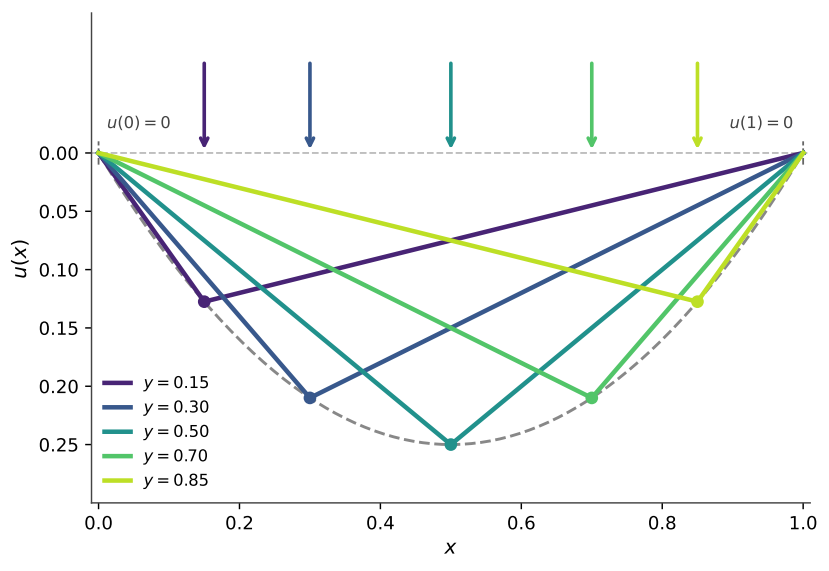
Column j of K is the tent for a load at y_j.

Learning K means learning N^2 entries. That is 16,384 numbers on 128 points and about a million on 1024 points.

Can a physical symmetry cut that number?

The shapes of the tents

Look at the tents again. A load near an end makes a short, lopsided tent. A load in the middle makes the tallest tent.



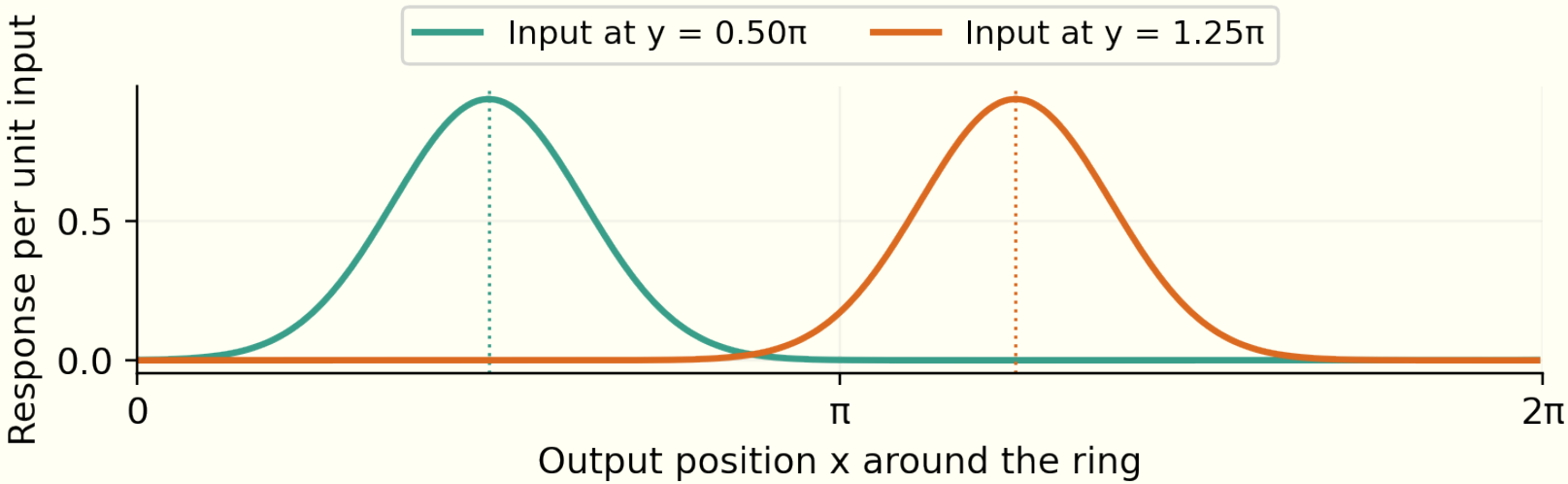
The pinned ends make every point of the string different. So every column of K is a new shape, and we must store all of them. What if every point of the domain looked the same?

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

Heat on a ring

Here is a thin metal ring with temperature $u(x)$ at the angle $x \in [0, 2\pi)$. Heat flows for a time T according to $u_t = \nu u_{xx}$, with $\nu = 0.14$ and $T = 0.65$. Every point of the ring looks the same, and heat flow on it has a solution we can write down. Put one unit of heat at the point y and let it flow.



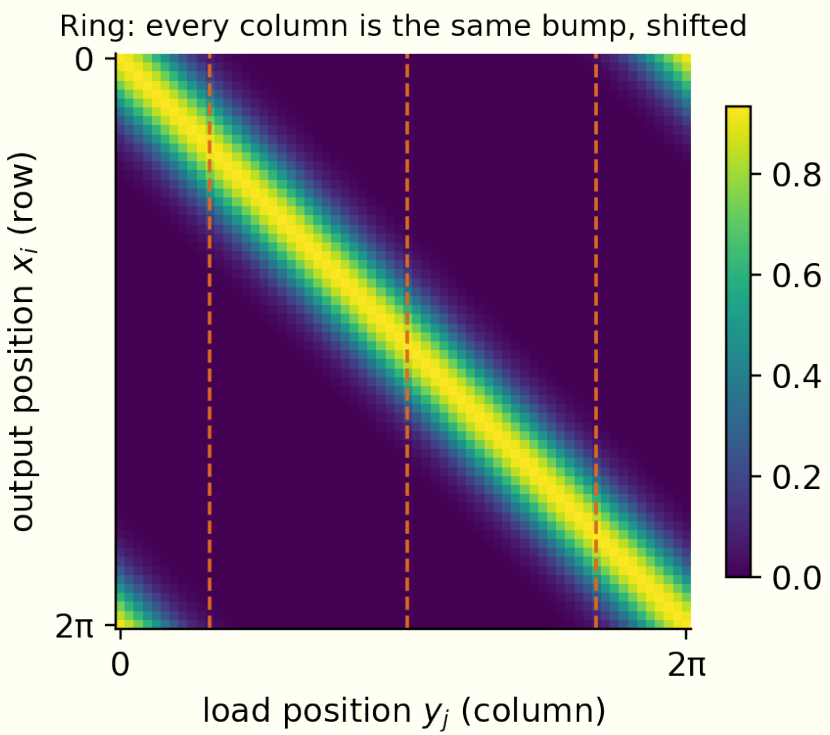
The heat spreads into a bump centered at y. Move y, and the bump moves with it and keeps its shape.

The ring's matrix

The ring has no ends, so the response at x depends only on how far x is from y,

$$\kappa(x, y) = \kappa(x - y),$$

with the separation $x - y$ measured around the ring.



Every column of the ring's matrix is the same bump, shifted down by one grid point. The bump wraps around at the bottom.

One column of N numbers now determines all N^2 entries. A matrix with this structure is **circulant**.

A convolution

Adding up point sources on the ring now gives

$$\boxed{u(x, T) = \int_0^{2\pi} \kappa(x - y)\, u_0(y)\, dy.}$$

This integral is the **convolution** of the initial temperature u_0 with the kernel κ. It slides one fixed shape κ around the ring and weights it by u_0.

A general Green's function depends on x and y separately. Boundaries, as on the string, and a conductivity that changes with x both prevent the convolution form. The Fourier layer is built for the convolution case.

A CNN filter on the ring

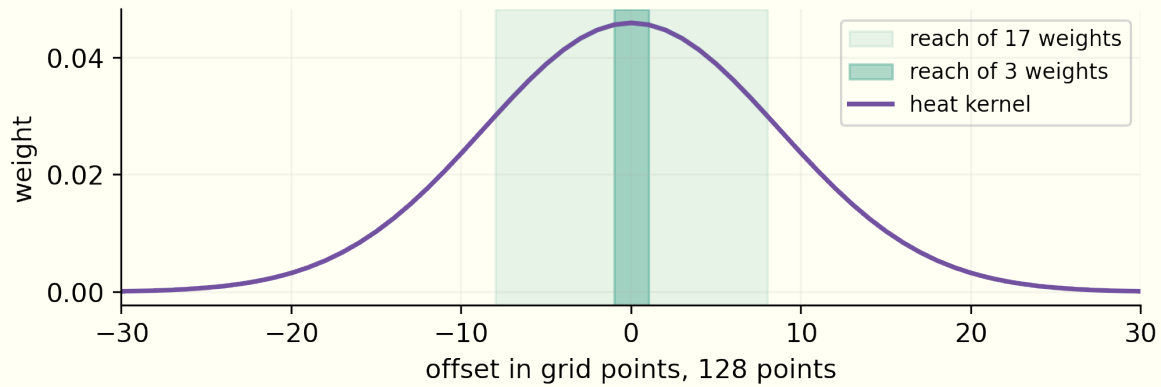
A CNN computes the same kind of sum with a short filter. The blur on a ring of six points is a circulant matrix,

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix}.$$

Each row is the row above, shifted one place. Shifting the input shifts the output, as on the ring.

The width of the heat kernel

The heat kernel is a Gaussian with standard deviation $\sqrt{2\nu T} = 0.43$. On 128 points the spacing is $2\pi/128 = 0.049$, so the kernel is 8.7 spacings wide.



CNN weights, fitted to 64 heat pairs	3	9	17	25
Held-out relative error	0.63	0.098	0.0035	0.00066

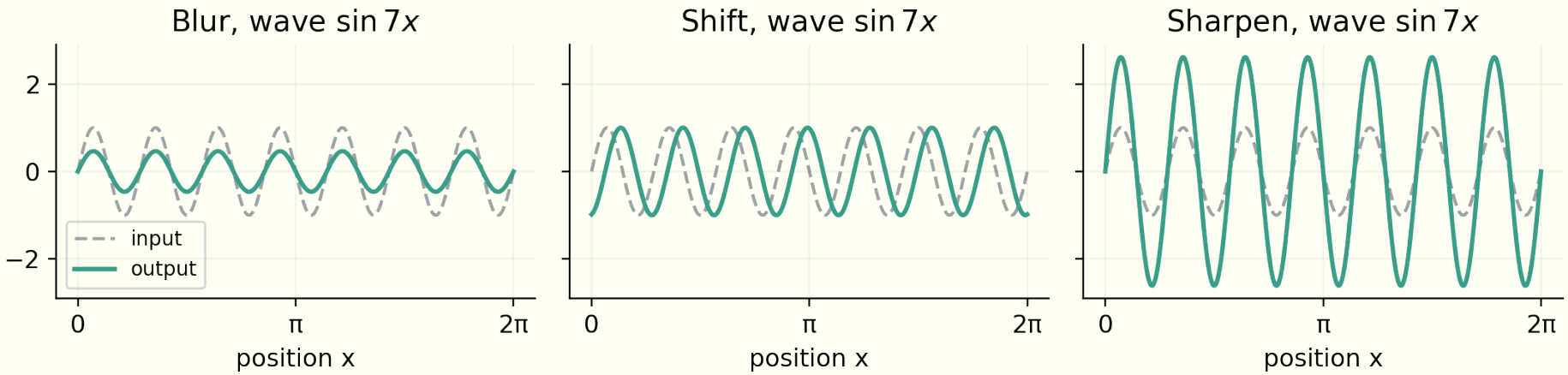
A wider filter works, but on 256 points the same kernel is 17.4 spacings wide. Can we describe the kernel with numbers that do not count grid points?

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

One wave through three filters

Try the simplest input, the single wave $\sin 7x$ on 32 points. Apply three image filters, the blur, the shift, and the sharpening filter.



The blur makes the wave smaller. The sharpening filter makes it larger. The shift moves it sideways. Every output is still a wave with seven periods, so no filter changed the frequency.

The multiplier of a wave

Write the wave as e^{ikx}. Moving it by s multiplies it by one number,

$$e^{ik(x-s)} = e^{-iks}\, e^{ikx}.$$

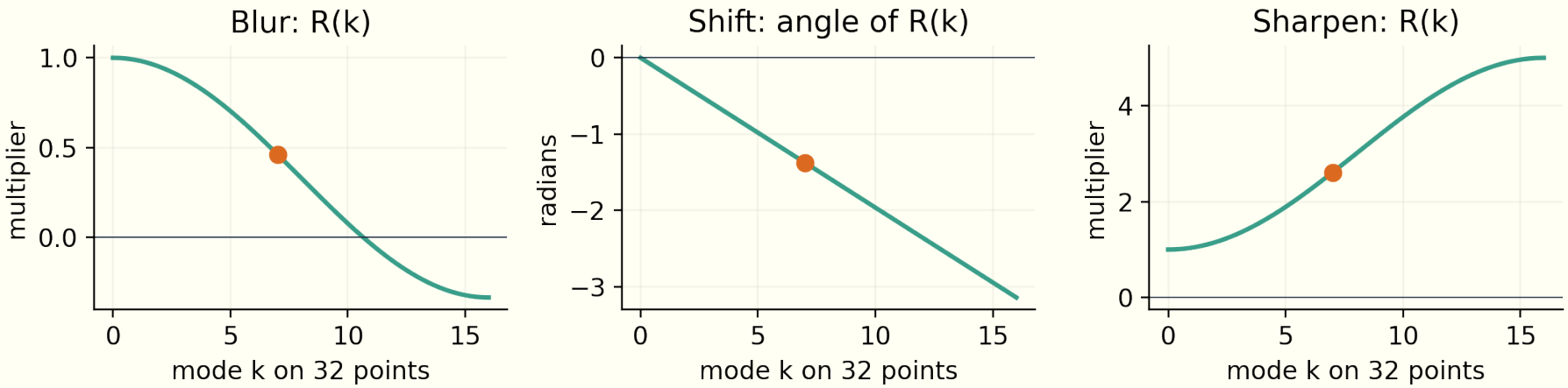
A convolution adds moved copies, so it also multiplies the wave by one number,

$$\int \kappa(s)\, e^{ik(x-s)}\, ds = \underbrace{\left(\int \kappa(s)\, e^{-iks}\, ds \right)}_{R(k)}\, e^{ikx}.$$

The number $R(k)$ is the **multiplier** of mode k.

Check it on the blur, which adds three copies moved by -1, 0, and 1 grid points. Its multiplier is $R(k) = \frac{1}{3}(1 + 2\cos\theta_k)$ with $\theta_k = 2\pi k/N$. For $\sin 7x$ on 32 points, $\theta_7 = 1.37$ and $\cos\theta_7 = 0.20$, so the blur multiplies the wave by 0.46.

The multipliers of three filters



The orange dots are the wave $\sin 7x$ from the last slide.

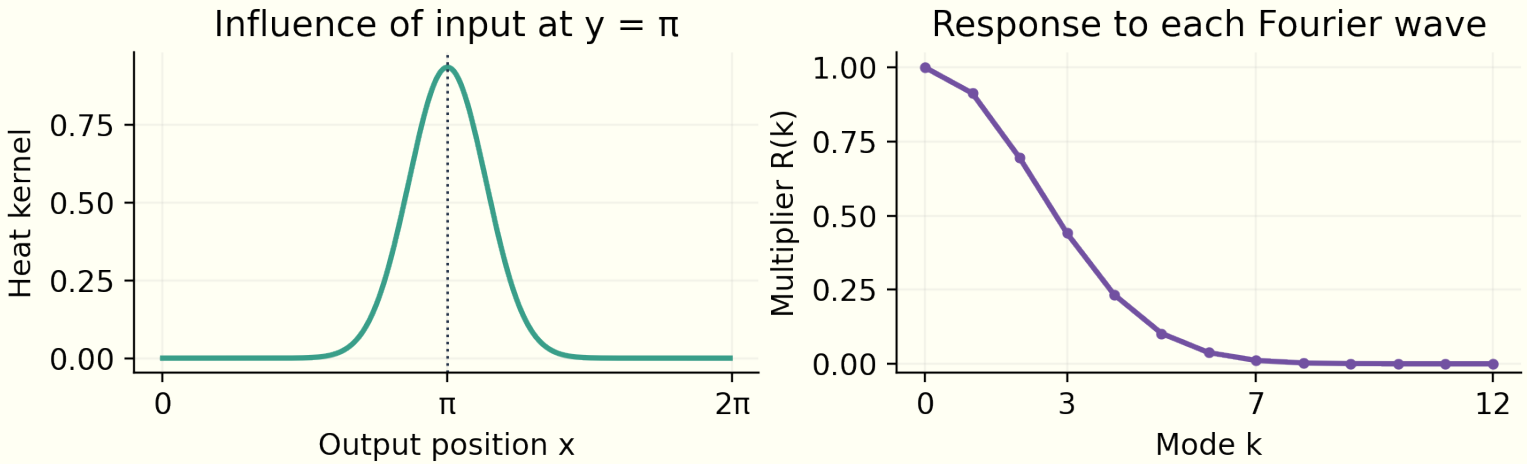
- The blur's multiplier falls with k, so it shrinks fast waves.
- The sharpening multiplier rises toward 5, so it strengthens fast waves.
- The shift keeps every size at 1 and turns the phase by θ_k.

All three multipliers equal 1 at $k = 0$, so all three filters leave a constant unchanged.

The multipliers of heat flow

Heat flow on the ring is a convolution, so it also multiplies each wave by one number. Put $u = q_k(t)\, e^{ikx}$ into $u_t = \nu u_{xx}$,

$$\frac{dq_k}{dt} = -\nu k^2 q_k, \qquad R(k) = e^{-\nu k^2 T}.$$



With $\nu T = 0.091$, the multiplier is 0.913 at $k = 1$, 0.441 at $k = 3$, and 0.0116 at $k = 7$. Fast waves have more curvature, so they decay first.

Fourier coordinates on the ring

A field on the ring is a sum of waves with amplitudes $\widehat{v}_k$,

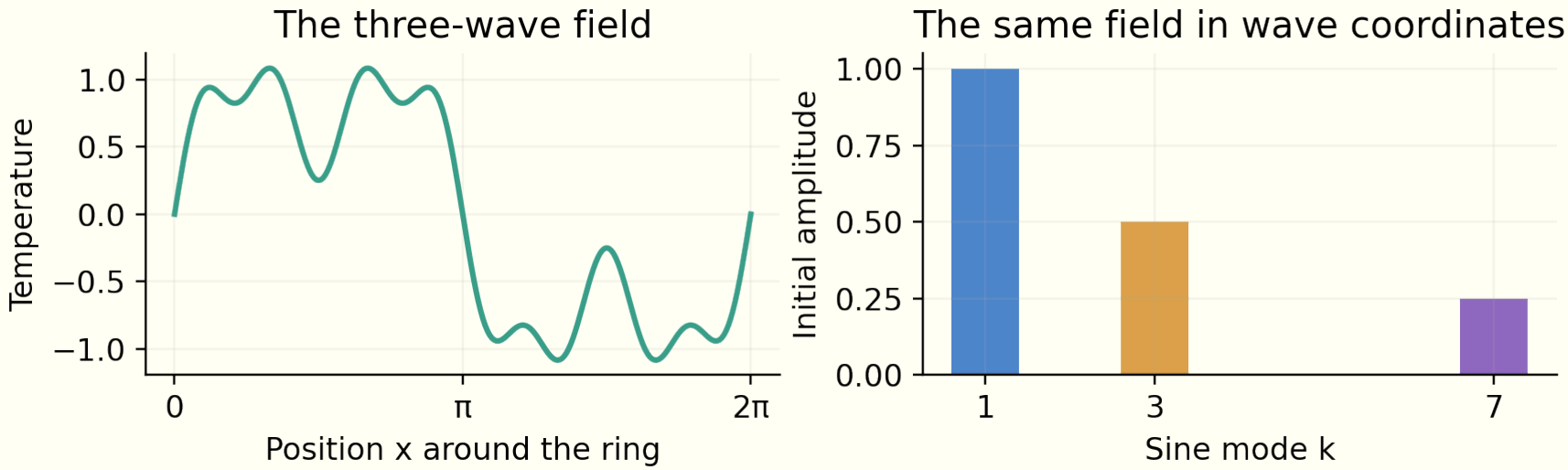
$$v(x) = \sum_{k \in \mathbb{Z}} \widehat{v}_k \, e^{ikx}, \qquad \widehat{v}_k = \frac{1}{2\pi} \int_0^{2\pi} v(x) \, e^{-ikx} \, dx.$$

On the grid $x_j = 2\pi j / N$, the FFT computes the amplitudes from the heights,

$$\widehat{v}_k = \frac{1}{N} \sum_{j=0}^{N-1} v(x_j) \, e^{-ikx_j}.$$

The heights $v(x_j)$ and the amplitudes $\widehat{v}_k$ are two lists that describe the same field. The FFT and its inverse convert one list into the other in $O(N \log N)$ operations.

Three waves



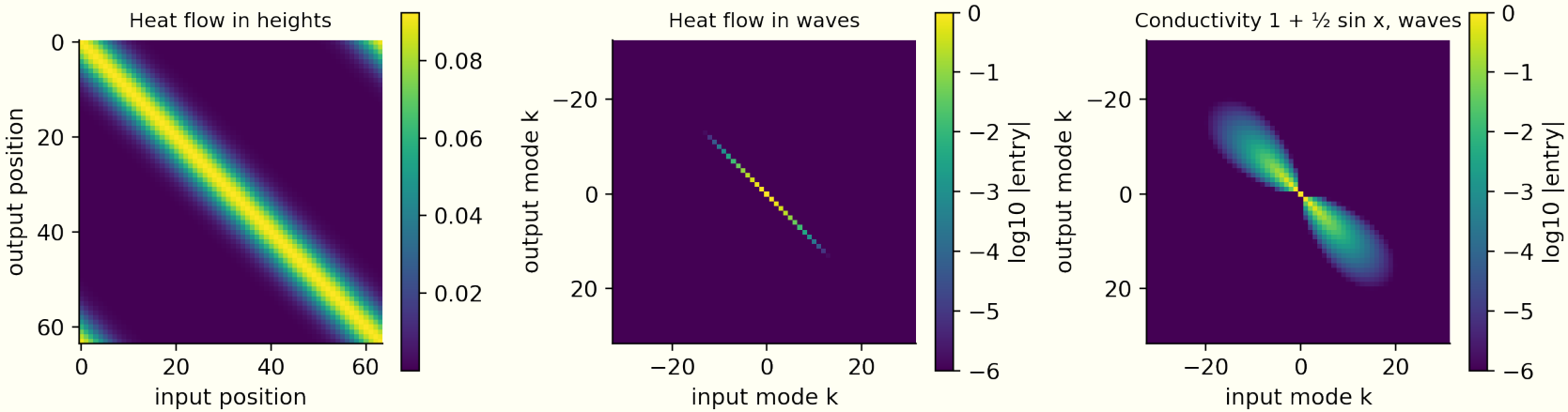
Take $u_0 = \sin x + \frac{1}{2}\sin 3x + \frac{1}{4}\sin 7x$. Heat flow multiplies each amplitude by its own number,

$$\begin{bmatrix} q_1(T) \\ q_3(T) \\ q_7(T) \end{bmatrix} = \mathrm{diag}(0.913,\, 0.441,\, 0.0116) \begin{bmatrix} 1 \\ 0.5 \\ 0.25 \end{bmatrix}.$$

The amplitude of mode 3 goes from 0.5 to $0.5 \times 0.441 \approx 0.220$.

One operator in two bases

The heights and the wave amplitudes are two coordinate systems for the same field. Choosing the Fourier waves is a change of basis, as choosing any other basis is. The basis alone does not make the Fourier layer. What matters is the matrix of the operator in that basis.



In heights, heat flow is a dense circulant matrix. In waves, the same operator is diagonal, so output mode k uses input mode k alone. A conductivity $1 + \frac{1}{2}\sin x$ breaks the shift symmetry, and the matrix in waves is no longer diagonal.

Convolution by FFT

A field is a sum of waves, and a convolution acts on each wave by itself. So three steps compute the convolution,

$$\boxed{u(\cdot, T) = \mathcal{F}^{-1}[R(k)\,\mathcal{F}(u_0)].}$$

1. The FFT finds the amplitude of each wave in u_0.
2. Multiply amplitude k by $R(k)$.
3. The inverse FFT adds the changed waves back together.

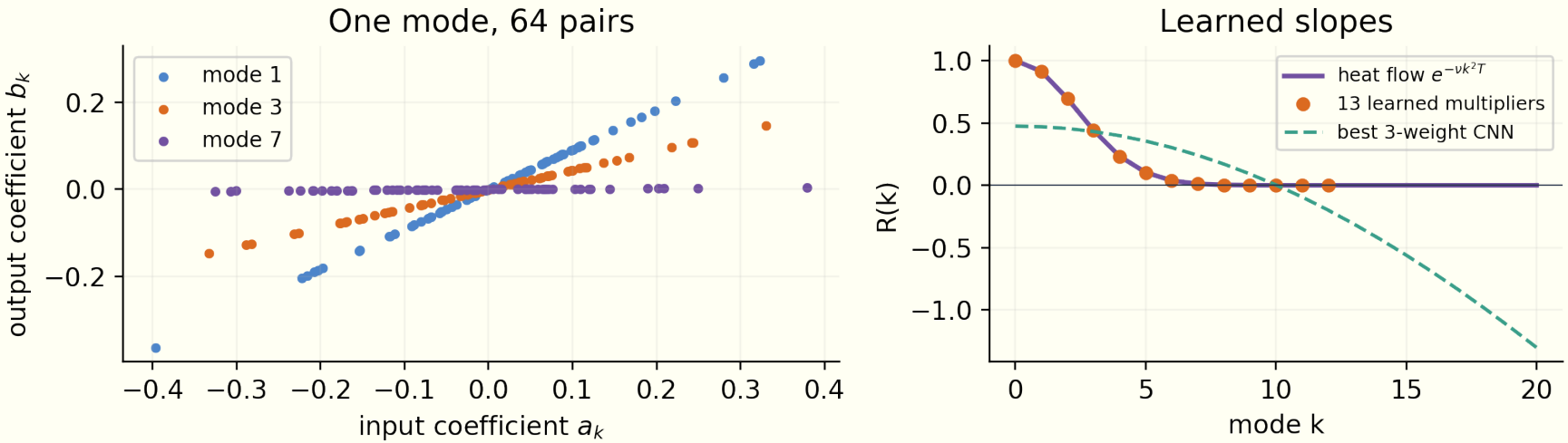
The kernel matrix has N^2 entries. The multipliers are $N/2 + 1$ numbers, and they refer to modes, not to grid points.

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

Multipliers from data

Now suppose we do not know ν. We have 64 pairs of initial and final profiles on 128 points, made of modes 0 to 12. For every pair j, mode k obeys $b_k^{(j)} = R(k)\, a_k^{(j)}$, so the pairs lie on a line through the origin.



Least squares fits the slope of each mode, $R(k) = \sum_j \overline{a_k^{(j)}}\, b_k^{(j)} / \sum_j |a_k^{(j)}|^2$. The 13 fitted numbers match $e^{-\nu k^2 T}$, and the held-out error is 2.2×10^{-16}. The best three-weight CNN is the dashed cosine, which cannot follow the heat curve.

The kernel of a Fourier layer

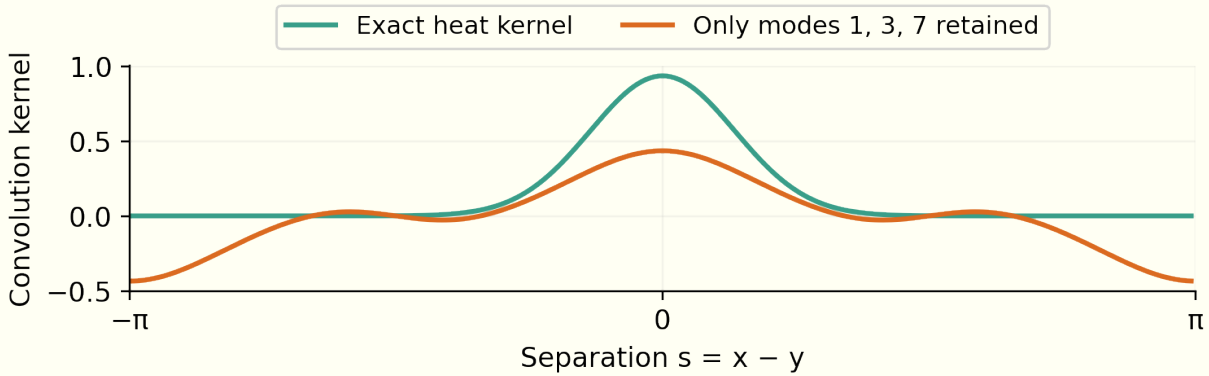
A Fourier layer keeps a set $\mathcal{M}$ of low modes and sets the others to zero,

$$\mathcal{K}v = \mathcal{F}^{-1}[R(k)\,\widehat{v}_k], \qquad k \in \mathcal{M}.$$

In space, the layer is a convolution with the kernel

$$\kappa(s) = \frac{1}{2\pi} \sum_{k \in \mathcal{M}} R(k)\, e^{iks},$$

and every retained wave extends around the whole ring.



With only modes 1, 3, and 7 kept, the kernel differs from the heat kernel. It gives the same output for every field made of those three modes.

Li et al., Fourier Neural Operator, Section 4.

Finer grids

A CNN's multiplier depends on the grid through $\theta_k = 2\pi k/N$. On 256 points, mode k sits where mode $k/2$ sat on 128 points. So a CNN trained on 128 points applies $e^{-\nu(k/2)^2 T} = e^{-\nu k^2 (T/4)}$, which is heat flow for the time $T/4$. The learned multiplier $R(k)$ is indexed by k itself.

Points	Fourier layer vs T	17-weight CNN vs T	17-weight CNN vs T/s
128	2.2×10^{-16}	0.0035	0.0035, $s = 1$
256	2.1×10^{-16}	0.78	0.0028, $s = 4$
512	2.2×10^{-16}	1.7	0.0021, $s = 16$

Both maps were trained once on 128 points. The transfer works because the test profiles contain only modes 0 to 12 and every grid resolves them.

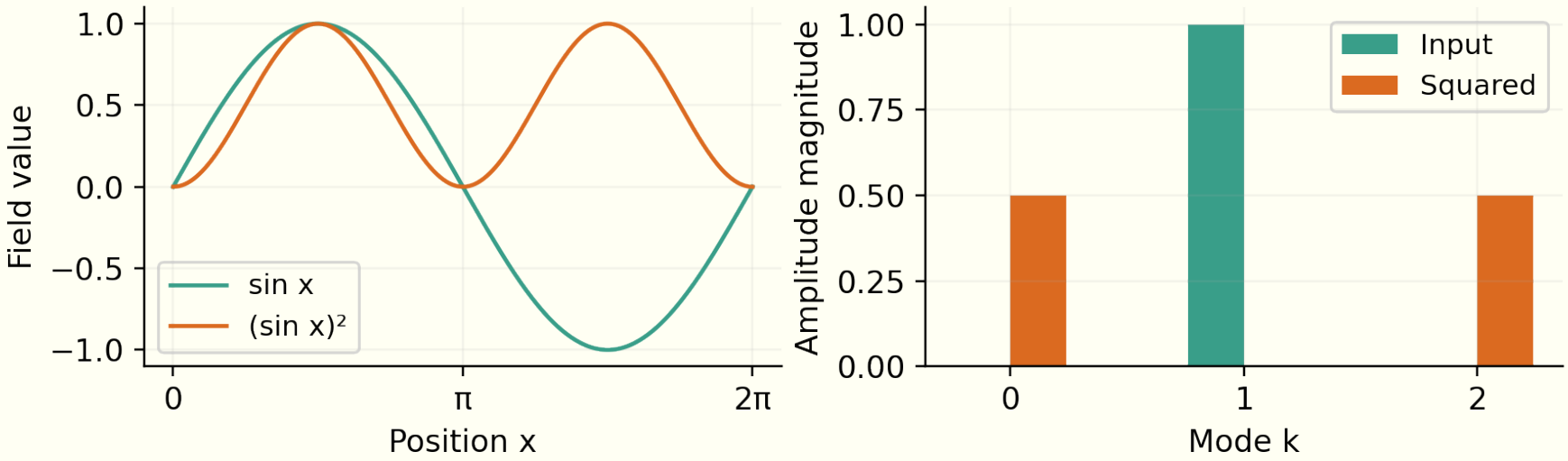
Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

Squaring a wave

Every map so far multiplies waves that are already in the input. Now square $\sin x$,

$$\sin^2 x = \frac{1}{2} - \frac{1}{2}\cos 2x.$$



The square contains modes 0 and 2, and the input contained only mode 1. A multiplier gives $R(2) \times 0 = 0$, so no multiplier can create mode 2. A pointwise nonlinear operation can.

One FNO block

An FNO block sends a feature field $v_\ell(x)$ down two paths and adds them,

$$
\begin{aligned}
z(x) &= \mathcal{F}^{-1}[R_\ell \, \mathcal{F}(v_\ell)](x) && \text{global, the learned multipliers} \\
y(x) &= W_\ell \, v_\ell(x) + b_\ell && \text{local, at each point} \\
v_{\ell+1}(x) &= \sigma(z(x) + y(x)) && \text{pointwise activation}
\end{aligned}
$$

- The spectral path multiplies mode k by $R_\ell(k)$, which couples every point of the domain.
- The local path multiplies every mode by the same matrix W_ℓ, and it keeps modes that the spectral cutoff drops.
- Both linear paths act on each mode separately. Only the activation σ couples different modes.

In two dimensions, k is a vector and the same formulas apply.

Channels

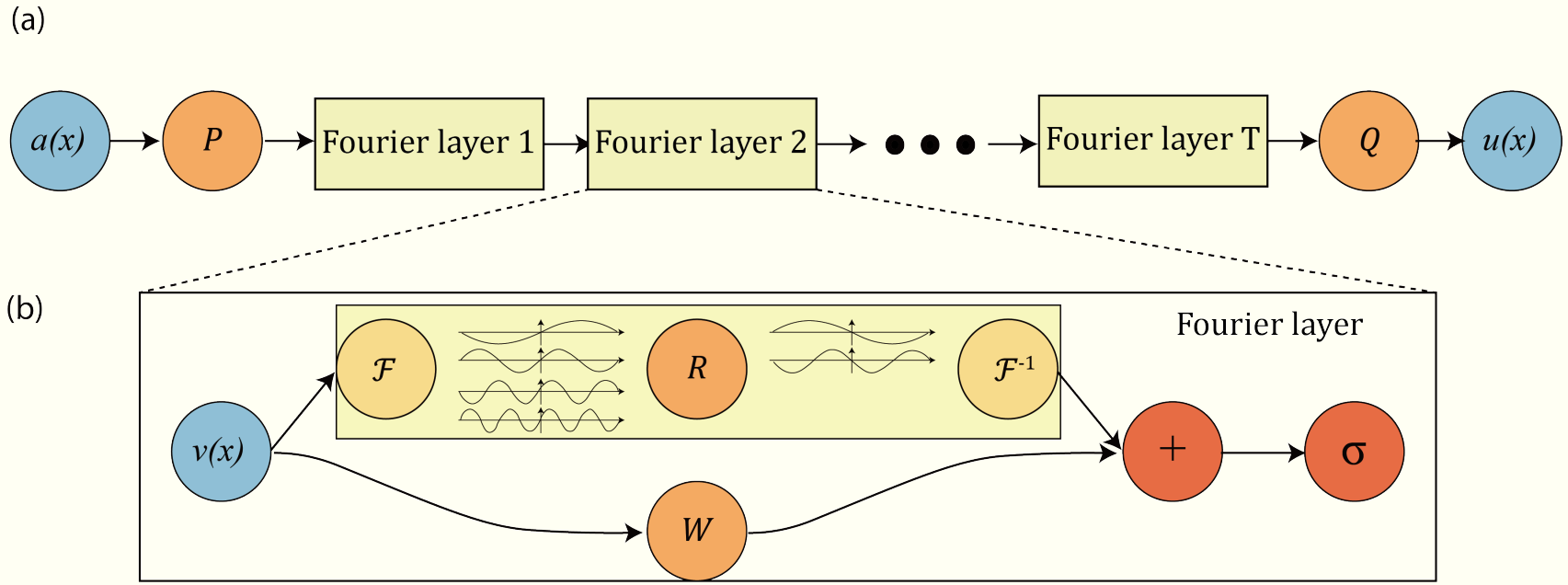
A feature field has C channels at each point, so $R_\ell(k)$ is a $C \times C$ matrix,

$$\widehat{z}_o(k) = \sum_{i=1}^{C} R_{oi}(k)\, \widehat{v}_i(k), \qquad k \in \mathcal{M}.$$

At a fixed mode k, the matrix mixes the channels. Different modes still do not mix.

Count the numbers in one 1D layer of the Burgers model, with $C = 64$ channels and 16 retained modes. The spectral path has $64 \times 64 \times 16 = 65{,}536$ complex multipliers. That count does not change when the grid is refined.

Complete FNO Architecture



Fourier Neural Operator architecture.

- The lifting P maps the input at each point to C channels, $v_0 = P(u)$.
- Four Fourier blocks compute $v_{\ell+1} = \sigma(W_\ell v_\ell + b_\ell + \mathcal{K}_\ell v_\ell)$.
- The projection Q maps the C channels at each point to the output.

Pseudospectral evaluation and an FNO block

Pseudospectral Burgers

1. Start with coefficients of u.
2. Apply known multipliers 1 and ik.
3. IFFT to values of u and u_x.
4. Form the product $u\,u_x$ on the grid.
5. FFT the product and integrate the PDE in time.

FNO features

1. Start with a feature field v_ℓ.
2. FFT and apply learned $R_\ell(k)$.
3. IFFT and add $W_\ell v_\ell + b_\ell$.
4. Apply a pointwise activation.
5. Continue to the next feature layer.

The solver forms the product $u\,u_x$ at the grid points, and the FNO applies an activation there. Both couple modes at the grid points, and FNO layers need not represent physical time steps.

McCabe et al., Section 2 and Appendix A.

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

The 1D Viscous Burgers Equation

Problem Formulation:
The 1D viscous Burgers equation combines nonlinear advection with diffusion,

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 2\pi], t \in [0, T]$$

with periodic boundary conditions and initial condition $u(x, 0) = u_0(x)$.

Operator learning task: Learn the mapping

$$\mathcal{G} : u_0(x) \mapsto u(x, T)$$

from initial condition to solution at time $T = 1$.

The data are 1000 training pairs and 100 test pairs with random initial conditions.

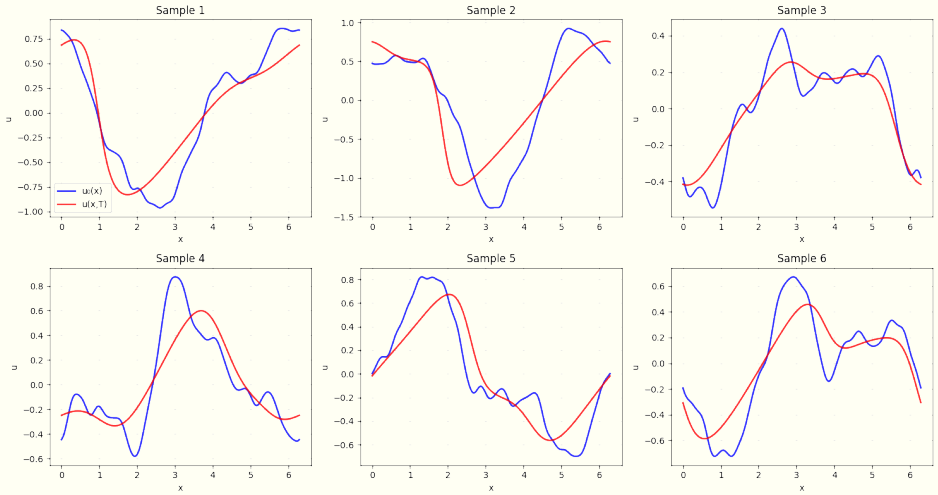
Burgers Equation: Physical Interpretation

The physics:

- **Nonlinear advection:** $u\frac{\partial u}{\partial x}$
- **Viscous diffusion:** $\nu\frac{\partial^2 u}{\partial x^2}$
- Advection steepens fronts, and viscosity keeps them smooth
- Periodic boundary conditions

Why this problem:

- Prototype for nonlinear PDEs
- Steep fronts need many Fourier modes
- Tests operator learning on complex dynamics



Sample initial conditions (blue) and evolved solutions (red) with steepened fronts.

1D FNO Architecture and Training

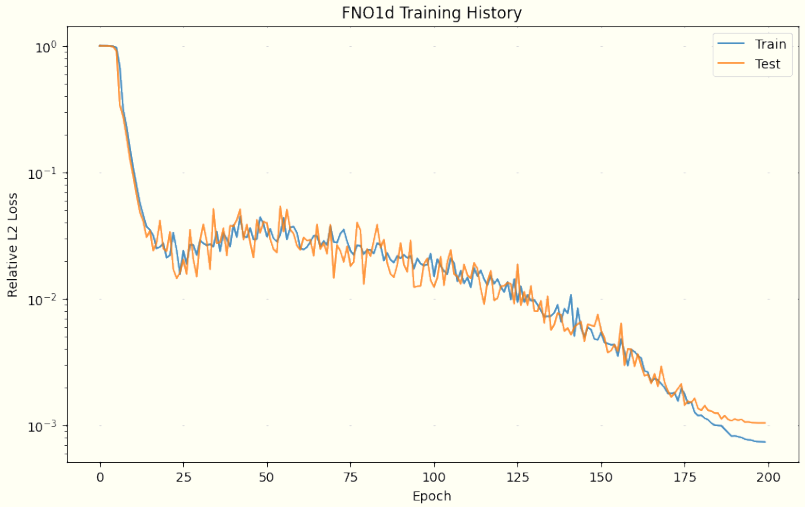
Model Configuration:

- **Input:** $(u_0(x), x)$ at $m = 2048$ points (subsampled to 512)
- **Fourier modes:** $k_{\max} = 16$
- **Hidden width:** $w = 64$
- **Layers:** 4 Fourier layers with skip connections
- **Activation:** GELU
- **Parameters:** $\sim$287K

Training Configuration:

- **Loss:** Relative L^2 loss: $\frac{\|u_{pred} - u_{true}\|_2}{\|u_{true}\|_2}$
- **Optimizer:** Adam with OneCycleLR scheduler
- **Training samples:** 1000
- **Test samples:** 100
- **Epochs:** 200

Burgers Equation: Training Results

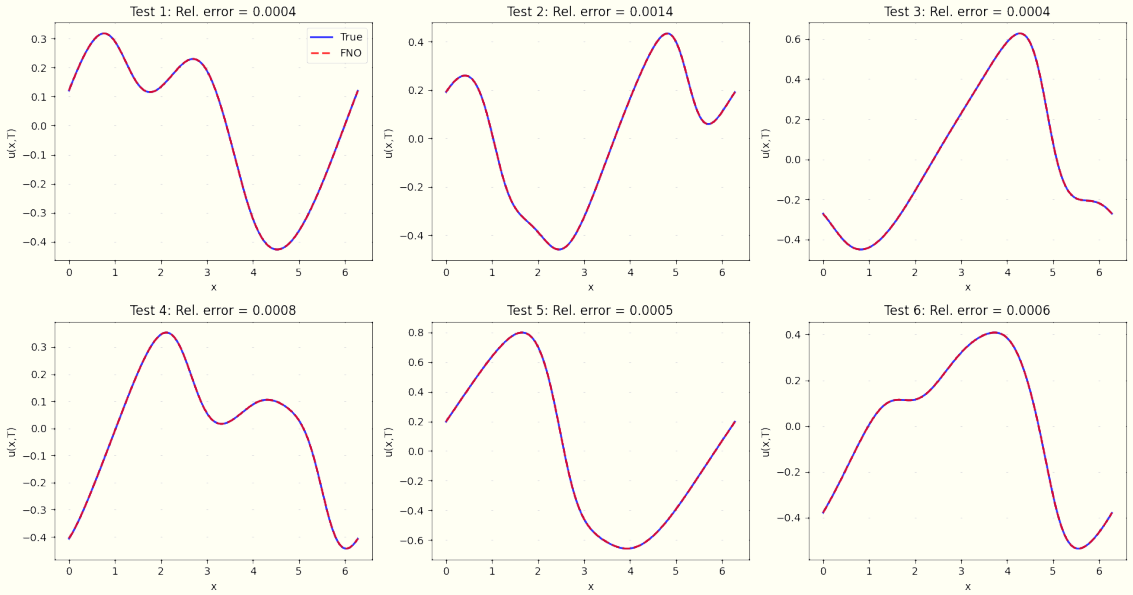


Training history showing convergence of relative L^2 loss.

Observations:

- Rapid convergence in first 50 epochs
- Final test error: $\sim$0.01-0.02 relative L^2
- No overfitting - train and test losses track together
- Model learns complex shock dynamics from data

Burgers Equation: Prediction Quality

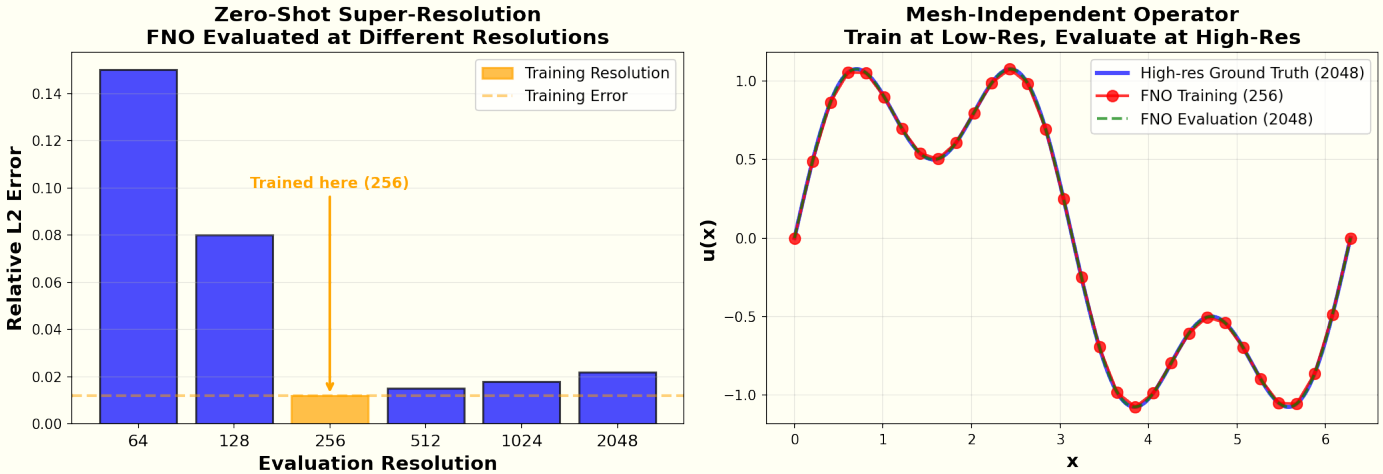


FNO predictions (red dashed) vs. true solutions (blue) for test cases.

Key results:

- Accurate prediction of shock locations and amplitudes
- Relative errors: 1-4%
- Works for initial conditions not seen during training
- Captures steep gradients despite spectral representation

Zero-Shot Super-Resolution



Training at 512 resolution, evaluating at 2048 resolution (4x upsampling).

Mesh Independence

- **Train:** 512 grid points
- **Test:** 2048 grid points (4x refinement)
- **Reported result:** Similar accuracy for this example

The same mode labels retain their meaning on the same physical domain. Shared parameters do not guarantee equal accuracy on every grid.

Outline

1. The Green's Function
2. A Ring with No Ends
3. Waves Through a Convolution
4. Learning the Multipliers
5. From One Layer to the FNO
6. Example 1: 1D Burgers Equation
7. Example 2: 2D Darcy Flow

The 2D Darcy Flow Equation

Problem Formulation:
The 2D Darcy flow equation models steady-state flow in porous media:

$$-\nabla \cdot (a(x,y)\nabla u(x,y)) = f(x,y), \quad (x,y) \in [0,1]^2$$

with zero boundary conditions: $u|_{\partial\Omega} = 0$.

Operator learning task: Learn the mapping

$$\mathcal{G} : a(x,y) \mapsto u(x,y)$$

from permeability coefficient a to pressure/hydraulic head u.

Physical interpretation:

- $a(x,y)$: permeability field (how easily fluid flows)
- $u(x,y)$: pressure field
- $f(x,y)$: source term (set to 1)

A variable coefficient changes the operator

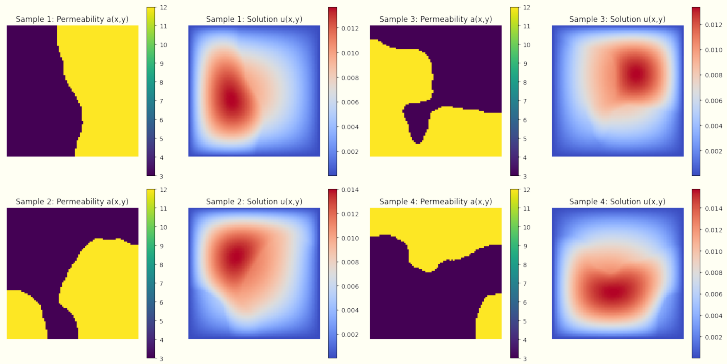
For each fixed permeability a, Darcy flow is linear in its forcing,

$$u(x) = \int_\Omega \kappa_a(x, y) f(y)\, dy.$$

The Green's function depends on a and the boundary conditions. It generally cannot be written as one fixed $\kappa(x - y)$.

- Multiplication by a spatially varying a couples Fourier modes.
- The learned task $a \mapsto u$ changes the PDE operator itself.
- Nonlinear feature layers and position information support this more general map.

Darcy Flow: Dataset and Physics



Left: Permeability fields $a(x, y)$ (input). Right: Pressure fields $u(x, y)$ (output).

Dataset characteristics:

- 1024 samples with random piecewise constant permeability
- Resolution: 241×241 (subsampled to 81×81)
- 1000 training samples, 100 test samples

Physical insight: Low permeability (dark) $\rightarrow$ high pressure gradients (red/yellow)

2D FNO Architecture

Model Configuration:

- **Input:** $(a(x, y), x, y)$ at 81×81 grid
- **Fourier modes:** $k_{\max} = 12$ (both x and y directions)
- **Hidden width:** $w = 32$
- **Layers:** 4 Fourier layers
- **Parameters:** $\sim$130K

Spectral Convolution 2D:

$$\text{FFT2} \rightarrow \text{Multiply modes } [:12, :12] \text{ and } [-12:, :12] \rightarrow \text{IFFT2}$$

Why two weight matrices? Due to symmetry of real FFT (rfft2), we need weights for both lower and upper frequencies.

Darcy Flow: Training Configuration

Training Setup:

- **Loss:** Relative L^2 loss in 2D
- **Optimizer:** Adam with step decay (StepLR)
- **Learning rate:** 0.001 with decay every 50 epochs
- **Batch size:** 20
- **Epochs:** 200 (500 in original paper)

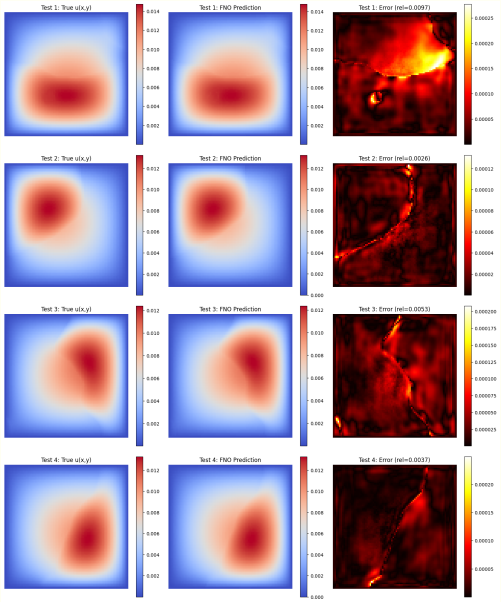
Data Normalization:

- Input permeability: Unit Gaussian normalization
- Output pressure: Unit Gaussian normalization
- Decode predictions before computing loss

Important

Normalize data for stable training, but always compute loss on physical quantities!

Darcy Flow: Results and Analysis



Left: True pressure. Middle: FNO prediction. Right: Error map.

Quantitative Results:

- Relative L^2 error: 2.9-3.2%
- Captures circular pressure distributions
- Handles heterogeneous permeability fields
- Structured errors (not random noise) suggest room for improvement

FNO properties

Shared mode weights

On the same physical domain, a retained wave number has the same meaning on different grids. Accuracy still depends on sampling, retained modes, and nonlinear aliasing.

Application cost

FFTs cost $O(N \log N)$ per fixed channel. Mode-wise channel transforms cost $O(MC_{\text{in}}C_{\text{out}})$. Training and reference-data generation add an upfront cost.

Global interaction

The learned convolution kernel can have global support even with a finite set of modes. Nonlinear blocks combine these global transformations with local operations.

Discretization, data, and physical constraints

Domain and grid

The standard FFT uses a regular grid and a periodic extension. Other boundaries and geometries need suitable representations.

Training coverage

Supervised learning needs reference input-output pairs. Unseen coefficients, forcing functions, or fine scales require evaluation.

Physical constraints

A standard prediction loss does not enforce the PDE or conservation laws. PDE-aware losses and explicit constraints can address particular requirements.

FNO vs DeepONet: Comparison

Aspect	DeepONet	FNO
Architecture	Branch-Trunk	Spectral Convolution
Space	Physical	Fourier
Queries	Arbitrary points	Grid points (FFT)
Input handling	Branch encoding	FFT on a uniform grid
Parameters	Branch/trunk dependent	Width/mode dependent
Global interaction	Learned basis coefficients	Spectral convolution
Output sampling	Coordinate queries	FFT grid in this model

Key difference:

- **DeepONet:** Learns basis decomposition in physical space
- **FNO:** Learns multiplication in Fourier space

The choice depends on input sampling, domain representation, and required output queries.

- Encode the boundary conditions and geometry explicitly
- Match the query pattern to the model implementation

Extensions to FNO

Geo-FNO: Arbitrary Geometries

- Use coordinate transforms to handle non-periodic domains
- Applies Fourier layers in transformed space
- Enables FNO for complex geometries

Physics-Informed FNO

- Add PDE residual to loss function
- Penalizes PDE residuals on the evaluated samples
- Reduces data requirements

Factorized FNO

- Low-rank approximations for 3D problems
- Reduces memory and computation
- Enables high-resolution 3D operator learning

Implementation Tips

1. Architecture Choices

- **Fourier modes:** Start with 12-16, adjust based on problem
- **Width:** 32-64 for most problems
- **Layers:** 4 is standard, more for complex operators
- **Activation:** GELU works well (smoother than ReLU)

2. Training Strategy

- **Loss:** Relative L^2 for scale-invariance
- **Optimizer:** Adam with learning rate scheduling
- **Normalization:** Unit Gaussian for inputs and outputs
- **Epochs:** 200-500 depending on problem complexity

3. Validation

- Test on different resolutions (super-resolution check)

Resources and Further Reading

Original Paper:

- Li et al., "Fourier Neural Operator for Parametric PDEs" (ICLR 2021)
- `https://arxiv.org/abs/2010.08895`

Extensions:

- Geo-FNO: `https://arxiv.org/abs/2207.05209`
- Physics-Informed Neural Operators: `https://arxiv.org/abs/2111.03794`
- Neural Operator Review: `https://arxiv.org/abs/2108.08481`

Code and Data:

- Official implementation: `https://github.com/neuraloperator/neuraloperator`
- Course notebooks: `https://github.com/kks32-courses/sciml`
- Dataset: `https://huggingface.co/datasets/kks32/sciml-dataset`

Questions?

Thank you!

Contact:

Krishna Kumar

krishnak@utexas.edu

University of Texas at Austin

Interactive Demo:

▸ FNO Notebook

Dataset:

▸ Hugging Face Dataset